

Adaptabilidad de un Sistema: Simulación de un Circuito RLC y Generalización a Sistemas Complejos

Santiago

Facultad de Ingeniería, Universidad Sergio Arboleda

Sistemas Complejos — SC_G01_2026_02

Bogotá, Colombia

Resumen— Este trabajo estudia el concepto de adaptabilidad de un sistema a partir de la simulación numérica de un circuito RLC en serie. Se derivó la representación en espacio de estados del circuito y se verificaron sus cuatro regímenes de amortiguamiento (no amortiguado, subamortiguado, críticamente amortiguado y sobreamortiguado) mediante el cálculo de los eigenvalores de la matriz de estados para distintos valores de resistencia R , manteniendo fijos L y C . A partir de estos resultados se argumenta qué se entiende por adaptabilidad de un sistema y se generaliza el mecanismo observado — un único parámetro que reconfigura el comportamiento cualitativo completo del sistema — a ejemplos de sistemas complejos reales como la termorregulación, los mercados económicos y las dinámicas depredador-presa.

Palabras clave—sistema RLC, adaptabilidad, sistemas complejos, espacio de estados, amortiguamiento, eigenvalores, retroalimentación.

I. INTRODUCCIÓN

Un sistema RLC en serie — una resistencia, un inductor y un capacitor conectados en un mismo lazo, con el capacitor inicialmente cargado — es uno de los ejemplos más simples de un sistema dinámico lineal de segundo orden. A pesar de su simplicidad, exhibe un fenómeno central en el estudio de los sistemas complejos: un único parámetro del sistema puede reconfigurar por completo su comportamiento cualitativo, pasando de una oscilación sostenida a una convergencia suave hacia el equilibrio. Este trabajo utiliza dicho circuito como modelo para argumentar qué se entiende por *adaptabilidad* de un sistema, y generaliza el mecanismo observado a sistemas complejos reales.

II. METODOLOGÍA

A. Ecuaciones del sistema

Aplicando la ley de voltajes de Kirchhoff al lazo RLC se obtiene, con vector de estado $\mathbf{x} = (q, i)^T$ — carga en el capacitor y corriente del lazo — un sistema lineal de primer orden:

$$\dot{q} = i, \quad L\dot{i} = -Ri - \frac{q}{C}, \quad (1)$$

es decir, $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x}$, con la matriz de estados

$$\mathbf{A} = \begin{bmatrix} 0 & 1 \\ -1/(LC) & -R/L \end{bmatrix}. \quad (2)$$

Los *estados* del sistema son, en cada instante, el par (carga, corriente). El comportamiento cualitativo de esos estados queda determinado por los eigenvalores de $\mathbf{A}$, que dependen de la razón de amortiguamiento

$$\omega_0 = \sqrt{\frac{1}{LC}}, \quad \zeta = \frac{R}{2} \sqrt{\frac{C}{L}}. \quad (3)$$

B. Los cuatro regímenes

- $\zeta = 0$ ($R = 0$): no amortiguado — oscila indefinidamente; eigenvalores puramente imaginarios.
- $0 < \zeta < 1$: subamortiguado — oscila mientras decae; eigenvalores complejos con parte real negativa.
- $\zeta = 1$: críticamente amortiguado — regresa al equilibrio en el menor tiempo posible, sin oscilar; eigenvalores reales iguales.
- $\zeta > 1$: sobreamortiguado — regresa al equilibrio lentamente, sin oscilar; eigenvalores reales distintos.

C. Verificación de los estados

Se simularon los cuatro regímenes con $L = 1$ H y $C = 0,25$ F fijos ($\omega_0 = 2$ rad/s, R crítico = 4Ω), variando únicamente R , con condición inicial $q_0 = 1$ C, $i_0 = 0$ A. Se integró el sistema con RK45 adaptativo (`scipy.integrate.solve_ivp`) y se calcularon los eigenvalores de $\mathbf{A}$ en cada caso para contrastar el régimen predicho por ζ contra el comportamiento numérico observado.

III. RESULTADOS

A. Regímenes temporales

La Fig. 1 muestra el voltaje en el capacitor $v_C(t) = q(t)/C$ y la corriente $i(t)$ para los cuatro regímenes.

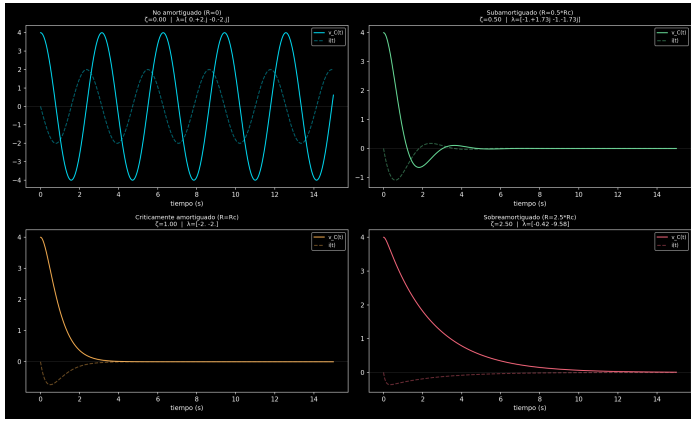


Figura 1. $v_C(t)$ e $i(t)$ para los cuatro regímenes, con los mismos L y C , variando solo R . Los eigenvalores λ mostrados confirman el régimen predicho por ζ .

B. Retrato de fase

La Fig. 2 presenta la trayectoria de cada régimen en el espacio de estados (q, i) .

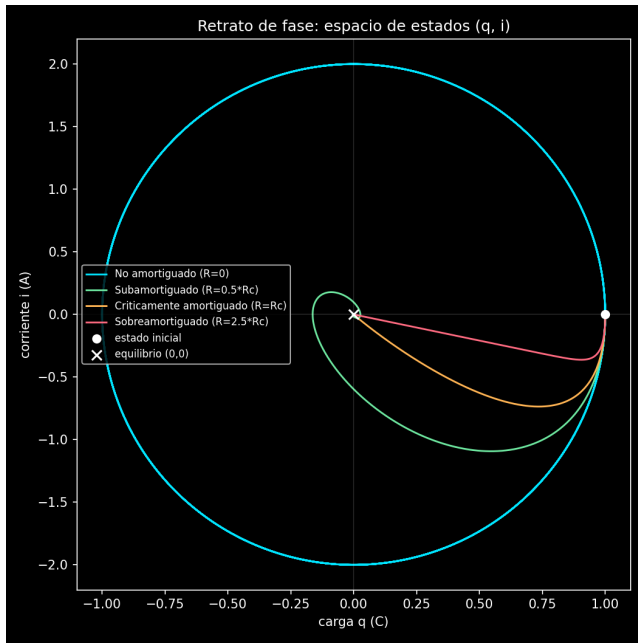


Figura 2. Retrato de fase. El caso no amortiguado traza un ciclo cerrado que nunca alcanza el equilibrio; los demás convergen hacia el origen $(0,0)$.

C. Verificación por eigenvalores

La Tabla 1 y la Fig. 3 resumen los eigenvalores de A obtenidos para cada régimen, confirmando en los cuatro casos una coincidencia exacta entre el régimen predicho analíticamente por ζ y el comportamiento numérico observado.

Cuadro 1. Verificación de los estados por eigenvalores de A

Régimen	$R (\Omega)$	ζ	Eigenvalores λ
No amortiguado	0.00	0.00	$\pm 2,00j$
Subamortiguado	2.00	0.50	$-1,00 \pm 1,73j$
Crít. amortiguado	4.00	1.00	$-2,00, -2,00$
Sobreamortiguado	10.00	2.50	$-0,42, -9,58$

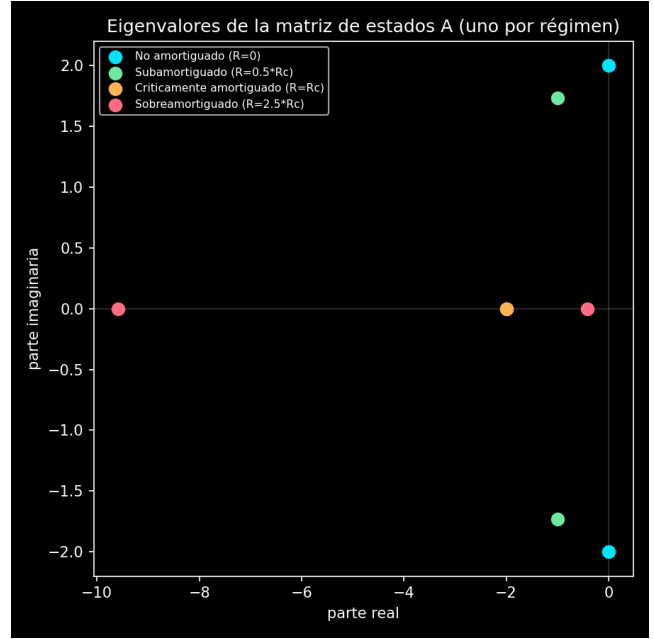


Figura 3. Eigenvalores de A en el plano complejo. Al aumentar R , los eigenvalores se desplazan del eje imaginario hacia el eje real negativo.

IV. ¿QUÉ SE ENTIENDE POR ADAPTABILIDAD DE UN SISTEMA?

La adaptabilidad de un sistema es su capacidad de ajustar su comportamiento interno — sin cambiar su estructura fundamental ni su objetivo — para mantener la estabilidad o alcanzar un nuevo equilibrio frente a perturbaciones del entorno. No equivale a ser simplemente estable: un sistema puede ser estable pero rígido (responde siempre igual, sin importar el contexto), mientras que un sistema adaptable modula su propia respuesta según las condiciones que enfrenta. Tres ingredientes suelen estar presentes en un sistema adaptable: (i) *sensado* del propio estado o de las perturbaciones recibidas; (ii) *retroalimentación*, de modo que la respuesta dependa del estado presente y no solo de una regla fija; y (iii) un *rango de operación ajustable*, es decir, uno o más parámetros internos capaces de reconfigurar el comportamiento cualitativo del sistema.

El circuito RLC no es, en sí mismo, adaptable — sus parámetros son fijos una vez fabricado — pero constituye un modelo idóneo del mecanismo subyacente: un único parámetro (R) determina si el sistema oscila indefinidamente, oscila decayendo, o converge al equilibrio de la forma más eficiente posible. Esa capacidad de un parámetro de reconfigurar el comportamiento cualita-

tivo completo del sistema es, precisamente, el tipo de mecanismo que — cuando ese parámetro puede ajustarse dinámicamente en respuesta al entorno — produce adaptabilidad genuina.

V. GENERALIZACIÓN A UN SISTEMA COMPLEJO

En un sistema complejo real, un mecanismo de retroalimentación cumple un papel análogo al de la resistencia R : actúa como un regulador que amortigua las desviaciones respecto a un punto de equilibrio, con la diferencia de que puede ajustarse dinámicamente según el estado del sistema, en lugar de permanecer fijo.

- **Termorregulación:** el sudor y el escalofrío modulan su intensidad según qué tan lejos esté la temperatura corporal del equilibrio (37°C), evitando tanto la oscilación descontrolada como la respuesta demasiado lenta.
- **Mercados económicos:** los bancos centrales ajustan tasas de interés — un parámetro de amortiguamiento de la economía — para evitar inflación oscilante o estancamiento.
- **Ecosistemas:** las poblaciones depredador-presa oscilan de forma sostenida (análogo al caso no amortiguado) salvo que existan mecanismos reguladores adicionales que actúen como amortiguamiento efectivo.

La generalización central es esta: en el circuito RLC, R es un valor fijo elegido externamente. En un sistema complejo verdaderamente adaptable, el propio sistema ajusta su “ R efectiva” en respuesta a su entorno o a su estado interno, recorriendo dinámicamente el mismo mapa de regímenes de la Fig. 3 en lugar de permanecer fijo en uno solo.

VI. CONCLUSIONES

La simulación confirmó experimentalmente los cuatro regímenes teóricos del sistema RLC mediante la verificación directa de los eigenvalores de su matriz de estados. A partir de este modelo se argumentó que la adaptabilidad de un sistema depende de que un parámetro capaz de reconfigurar el comportamiento cualitativo del sistema pueda ajustarse dinámicamente — mediante sensado y retroalimentación — en lugar de permanecer fijo, mecanismo que se generalizó a ejemplos de sistemas complejos biológicos, económicos y ecológicos.

APÉNDICE — CÓDIGO FUENTE (RLC_SIMULATION.PY)

```
1 """
2 Simulacion de un sistema RLC en serie - verificacion de sus estados
3 Sistemas Complejos - Taller: Adaptabilidad
4
5 Circuito RLC serie (R, L, C en serie con un capacitor cargado inicialmente):
6
7  $L \cdot di/dt + R \cdot i + q/C = 0$   $dq/dt = i$ 
8
9 Representacion en espacio de estados, con  $x = [q, i]^T$ :
10
11  $dx/dt = A x$ ,  $A = \begin{bmatrix} 0 & 1 \\ -1/(LC) & -R/L \end{bmatrix}$ 
12
13 Los "estados" del sistema (en el sentido de teoria de sistemas) son el par
14 (carga  $q$ , corriente  $i$ ) en cada instante. El COMPORTAMIENTO de esos estados
15 (si oscilan, si decaen suavemente, si regresan lo mas rapido posible al
16 equilibrio) esta determinado por los valores propios (eigenvalores) de  $A$ ,
17 que a su vez dependen de la razon de amortiguamiento zeta:
18
19  $\omega_0 = \sqrt{1/(L \cdot C)}$  frecuencia natural
20  $\zeta = (R/2) \cdot \sqrt{C/L}$  razon de amortiguamiento
21
22  $\zeta == 0$  -> no amortiguado (oscila para siempre)
23  $0 < \zeta < 1$  -> subamortiguado (oscila decayendo)
24  $\zeta == 1$  -> criticamente amort. (vuelve mas rapido sin oscilar)
25  $\zeta > 1$  -> sobreamortiguado (vuelve lento, sin oscilar)
26 """
27
28 import numpy as np
29 import matplotlib.pyplot as plt
30 from scipy.integrate import solve_ivp
31
32 plt.style.use('dark_background')
33
34 # -----
35 # 1. PARAMETROS FIJOS DEL CIRCUITO (L, C) Y ESTADO INICIAL
36 # -----
37 L = 1.0 # henrios
38 C = 0.25 # faradios
39 Q0 = 1.0 # carga inicial en el capacitor (culombios)
40 I0 = 0.0 # corriente inicial (amperios)
41
42 OMEGA0 = np.sqrt(1 / (L * C))
43 R_CRITICO = 2 * np.sqrt(L / C) # R que produce  $\zeta = 1$ 
44
45 print(f"omega0 = {OMEGA0:.3f} rad/s")
46 print(f"R critico ( $\zeta=1$ ) = {R_CRITICO:.3f} ohm")
47
48 # -----
49 # 2. MODELO EN ESPACIO DE ESTADOS
50 # -----
51
52 def state_matrix(R, L, C):
53     return np.array([[0, 1],
54                      [-1 / (L * C), -R / L]])
55
56
57 def rlc_derivatives(t, x, R, L, C):
58     A = state_matrix(R, L, C)
59     return A @ x
60
61
62 def simulate(R, L, C, x0, t_max=15, n_steps=3000):
63     t_eval = np.linspace(0, t_max, n_steps)
64     sol = solve_ivp(rlc_derivatives, [0, t_max], x0, t_eval=t_eval,
65                     args=(R, L, C), method='RK45', rtol=1e-9, atol=1e-9)
66     return sol.t, sol.y[0], sol.y[1] # t, q(t), i(t)
67
68
69 def damping_ratio(R, L, C):
70     return (R / 2) * np.sqrt(C / L)
71
72
73 def classify_regime(zeta):
74     if zeta == 0:
75         return "No amortiguado"
```

```

76     elif zeta < 1:
77         return "Subamortiguado"
78     elif zeta == 1:
79         return "Críticamente amortiguado"
80     else:
81         return "Sobreamortiguado"
82
83
84 # -----
85 # 3. VERIFICACION DE LOS ESTADOS: EIGENVALORES DE LA MATRIZ A
86 # -----
87 def verify_states(R, L, C):
88     A = state_matrix(R, L, C)
89     eigvals = np.linalg.eigvals(A)
90     zeta = damping_ratio(R, L, C)
91     regime = classify_regime(zeta)
92     return eigvals, zeta, regime
93
94
95 # -----
96 # 4. ESCENARIOS: LOS 4 REGIMENES, MISMOS L Y C, VARIANDO R
97 # -----
98 SCENARIOS = {
99     "No amortiguado (R=0)": 0.0,
100     "Subamortiguado (R=0.5*Rc)": 0.5 * R_CRITICO,
101     "Críticamente amortiguado (R=Rc)": R_CRITICO,
102     "Sobreamortiguado (R=2.5*Rc)": 2.5 * R_CRITICO,
103 }
104 COLORS = ['#00e5ff', '#6ee7a0', '#ffb454', '#ff6b81']
105
106
107 def run_all():
108     results = {}
109
110     fig_time = plt.figure(figsize=(15, 9))
111     fig_phase = plt.figure(figsize=(7, 7))
112     ax_phase = fig_phase.add_subplot(1, 1, 1)
113
114     for idx, (name, R) in enumerate(SCENARIOS.items()):
115         t, q, i = simulate(R, L, C, [Q0, I0])
116         vC = q / C # voltaje en el capacitor
117
118         eigvals, zeta, regime = verify_states(R, L, C)
119         results[name] = {"R": R, "zeta": zeta, "regime": regime,
120                         "eigvals": eigvals}
121
122         # --- voltaje en el capacitor vs tiempo ---
123         ax1 = fig_time.add_subplot(2, 2, idx + 1)
124         ax1.plot(t, vC, color=COLORS[idx], label='v_C(t)')
125         ax1.plot(t, i, color=COLORS[idx], alpha=0.45, linestyle='--', label='i(t)')
126         ax1.set_title(f"{name}\nζ={zeta:.2f} | λ={np.round(eigvals,2)}",
127                      fontsize=9.5)
128         ax1.set_xlabel("tiempo (s)")
129         ax1.legend(fontsize=8, loc='upper right')
130         ax1.axhline(0, color='white', alpha=0.15, lw=0.8)
131
132         # --- retrato de fase (espacio de estados) ---
133         ax_phase.plot(q, i, color=COLORS[idx], label=name, lw=1.4)
134
135     fig_time.tight_layout()
136     fig_time.savefig('/home/claude/rlc/rlc_regimenes_temporales.png', dpi=150,
137                     facecolor='black')
138
139     ax_phase.scatter([Q0], [I0], color='white', zorder=5, s=40,
140                     label='estado inicial')
141     ax_phase.scatter([0], [0], color='white', marker='x', zorder=5, s=60,
142                     label='equilibrio (0,0)')
143     ax_phase.set_xlabel('carga q (C)')
144     ax_phase.set_ylabel('corriente i (A)')
145     ax_phase.set_title('Retrato de fase: espacio de estados (q, i)')
146     ax_phase.legend(fontsize=8)
147     ax_phase.axhline(0, color='white', alpha=0.15, lw=0.8)
148     ax_phase.axvline(0, color='white', alpha=0.15, lw=0.8)
149     fig_phase.tight_layout()
150     fig_phase.savefig('/home/claude/rlc/rlc_retrato_fase.png', dpi=150,
151                      facecolor='black')
152

```

```

153 # --- mapa de eigenvalores en el plano complejo ---
154 fig_eig, ax_eig = plt.subplots(figsize=(6, 6))
155 for idx, (name, R) in enumerate(SCENARIOS.items()):
156     eigvals = results[name]["eigvals"]
157     ax_eig.scatter(eigvals.real, eigvals.imag, color=COLORS[idx],
158                   s=70, label=name, zorder=5)
159 ax_eig.axhline(0, color='white', alpha=0.2, lw=0.8)
160 ax_eig.axvline(0, color='white', alpha=0.2, lw=0.8)
161 ax_eig.set_xlabel('parte real')
162 ax_eig.set_ylabel('parte imaginaria')
163 ax_eig.set_title('Eigenvalores de la matriz de estados A (uno por régimen)')
164 ax_eig.legend(fontsize=7.5, loc='upper left')
165 fig_eig.tight_layout()
166 fig_eig.savefig('/home/claude/rlc/rlc_eigenvalores.png', dpi=150)
167
168 return results
169
170
171 if __name__ == "__main__":
172     results = run_all()
173     print("\n=== VERIFICACION DE LOS ESTADOS (eigenvalores de A) ===\n")
174     print(f"{'Escenario':<35}{'R (ohm)':<10}{'zeta':<8}{'Eigenvalores':<28}{'Regimen'}")
175     for name, r in results.items():
176         ev_str = ", ".join(f"{e:.2f}" for e in r['eigvals'])
177         print(f"{name:<35}{r['R']:<10.3f}{r['zeta']:<8.2f}{ev_str:<28}{r['regime']}")

```